

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №3

Тестирование API средствами Postman

Выполнил:

Катков Алексей Сергеевич

Группа К3339

Поток БР1.2

Проверил:

Добряков Давид Ильич

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование API средствами Postman и написать автотесты внутри Postman. В результате необходимо получить один комплексный сценарий по основному процессу в приложении.

В рамках работы тестировался API сервиса обмена рецептами и кулинарных блогов. Основной пользовательский сценарий был выбран следующим образом:

регистрация пользователя → авторизация → создание рецепта → получение списка рецептов с фильтрацией → получение рецепта по идентификатору.

Ход работы

Описание тестируемого API

Тестируемое приложение представляет собой серверную часть сервиса для публикации и просмотра кулинарных рецептов. В проекте используются Node.js, Express, TypeScript, PostgreSQL и TypeORM. Для авторизации пользователей применяется JWT-токен.

Для проверки API была создана коллекция Postman Д33 - Recipe Sharing API - Postman Tests. В коллекции были выделены две основные группы запросов:

- 01 Auth - регистрация и авторизация пользователя;
- 02 Recipes - создание и получение рецептов.

Структура коллекции представлена на рисунке 1.

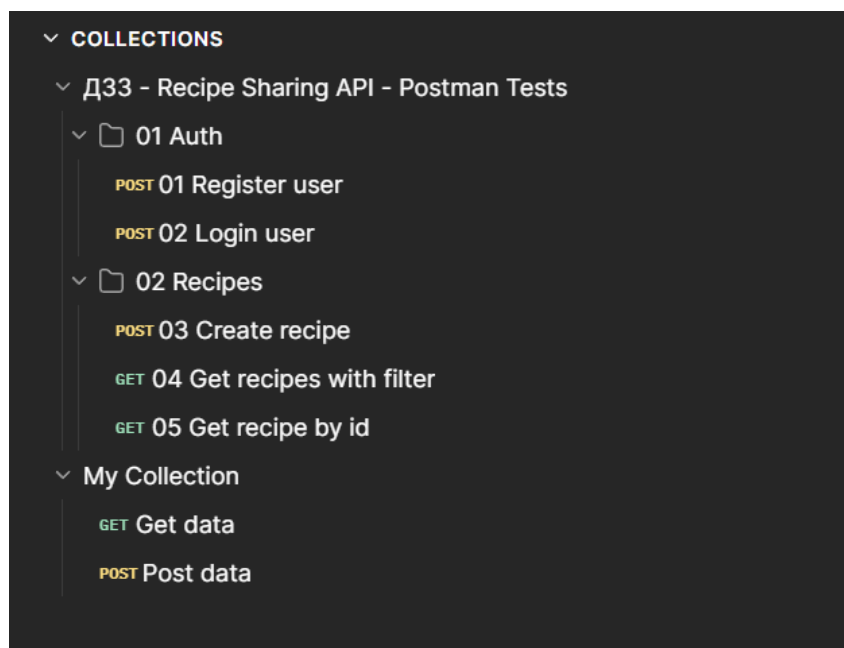


Рис. 1: Структура коллекции Postman

Настройка окружения Postman

Для удобного запуска тестов было создано окружение Recipe Sharing API Local Environment. В окружении использовались переменные:

Переменная	Назначение
baseUrl	Базовый URL локального API: http://localhost:8000/api
token	JWT-токен, полученный после авторизации пользователя
userId	Идентификатор авторизованного пользователя
recipeId	Идентификатор рецепта, созданного во время выполнения сценария

Переменные token, userId и recipeId заполняются автоматически в ходе выполнения запросов. Это позволяет связывать запросы между собой и запускать коллекцию как единый сценарий.

Сценарий тестирования

В итоговый сценарий вошли следующие запросы:

№	Запрос	Метод и URL	Назначение
1	Register user	POST /auth/register	Регистрация тестового пользователя
2	Login user	POST /auth/login	Авторизация пользователя и получение JWT-токена
3	Create recipe	POST /recipes	Создание нового рецепта от имени пользователя
4	Get recipes with filter	GET /recipes?difficulty=easy	Получение списка рецептов с фильтром по сложности
5	Get recipe by id	GET /recipes/{recipeId}	Получение информации о рецепте по идентификатору

Регистрация пользователя

Первым шагом выполняется регистрация пользователя. В теле запроса передаются имя, фамилия, имя пользователя, email и пароль.

Для запроса регистрации были добавлены проверки статуса ответа, формата JSON и наличия сообщения в ответе сервера.

Listing 1: Пример тестов для запроса регистрации

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

const response = pm.response.json();

pm.test("Response is JSON object", function () {
  pm.expect(response).to.be.an("object");
});

pm.test("Response contains message", function () {
  pm.expect(response.message).to.exist;
});
```

Авторизация пользователя

После регистрации выполняется вход пользователя в систему. В результате сервер возвращает `accessToken` и объект пользователя. Полученный токен сохраняется в переменную коллекции и используется в последующих запросах.

Listing 2: Сохранение токена и идентификатора пользователя

```
const response = pm.response.json();

pm.test("Access token exists", function () {
  pm.expect(response.accessToken).to.exist;
});

pm.test("User id exists", function () {
  pm.expect(response.user.id).to.exist;
});

pm.collectionVariables.set("token", response.accessToken);
pm.collectionVariables.set("userId", response.user.id);
```

На рисунке 2 показан результат выполнения запроса авторизации. В ответе присутствуют JWT-токен и данные пользователя.

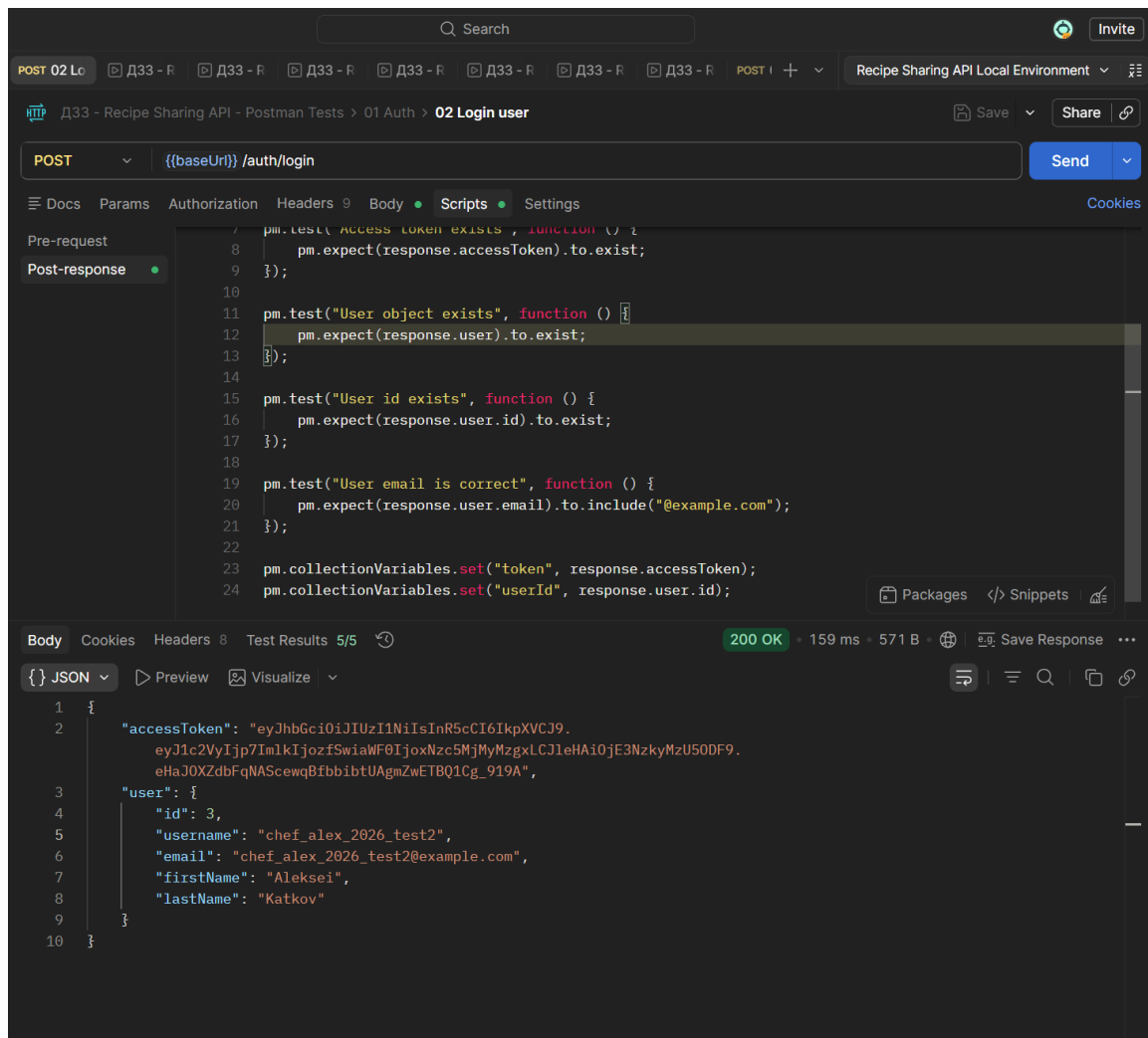


Рис. 2: Запрос авторизации пользователя и получение JWT-токена

Создание рецепта

После авторизации выполняется создание рецепта. В тело запроса передаются название, описание, сложность, время приготовления, количество порций и идентификатор автора.

Для запроса создания рецепта проверяется:

- успешный HTTP-статус;
- наличие идентификатора рецепта;
- корректность названия рецепта;
- корректность сложности рецепта;
- наличие автора рецепта.

Также идентификатор созданного рецепта сохраняется в переменную `recipeId`.

Listing 3: Тесты для запроса создания рецепта

```
const response = pm.response.json();

pm.test("Status code is 200 or 201", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

pm.test("Recipe has id", function () {
  pm.expect(response.id).to.exist;
});

pm.test("Recipe title is correct", function () {
  pm.expect(response.title).to.eql("Паста с томатным соусом");
});

pm.test("Recipe difficulty is easy", function () {
  pm.expect(response.difficulty).to.eql("easy");
});

pm.collectionVariables.set("recipeId", response.id);
```

Результат создания рецепта представлен на рисунке 3.

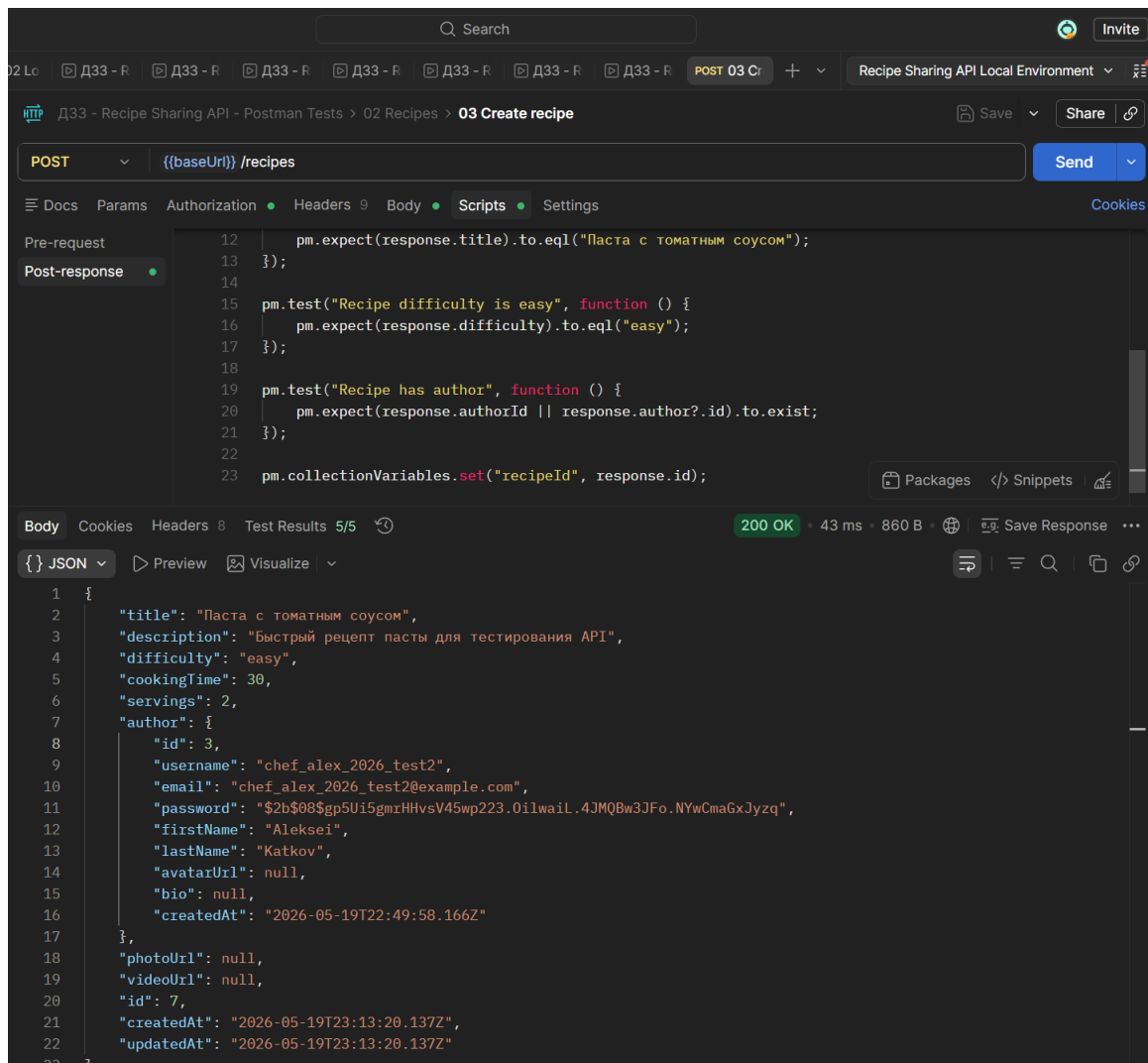


Рис. 3: Создание рецепта через Postman

Получение списка рецептов с фильтрацией

Следующим шагом выполняется запрос списка рецептов с фильтрацией по сложности `easy`. Для запроса проверяется статус ответа, наличие ответа, корректность типа данных и наличие созданного рецепта в ответе.

Listing 4: Тесты для получения списка рецептов

```

pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

const response = pm.response.json();

pm.test("Response exists", function () {
  pm.expect(response).to.exist;
}

```

```

});

pm.test("Response is array or object", function () {
    pm.expect(response).to.satisfy(function (data) {
        return Array.isArray(data) || typeof data === "object";
    });
});

pm.test("Response contains created recipe data", function () {
    const text = JSON.stringify(response);
    pm.expect(text).to.include("Паста с томатным соусом");
});

```

Результат выполнения запроса показан на рисунке 4.

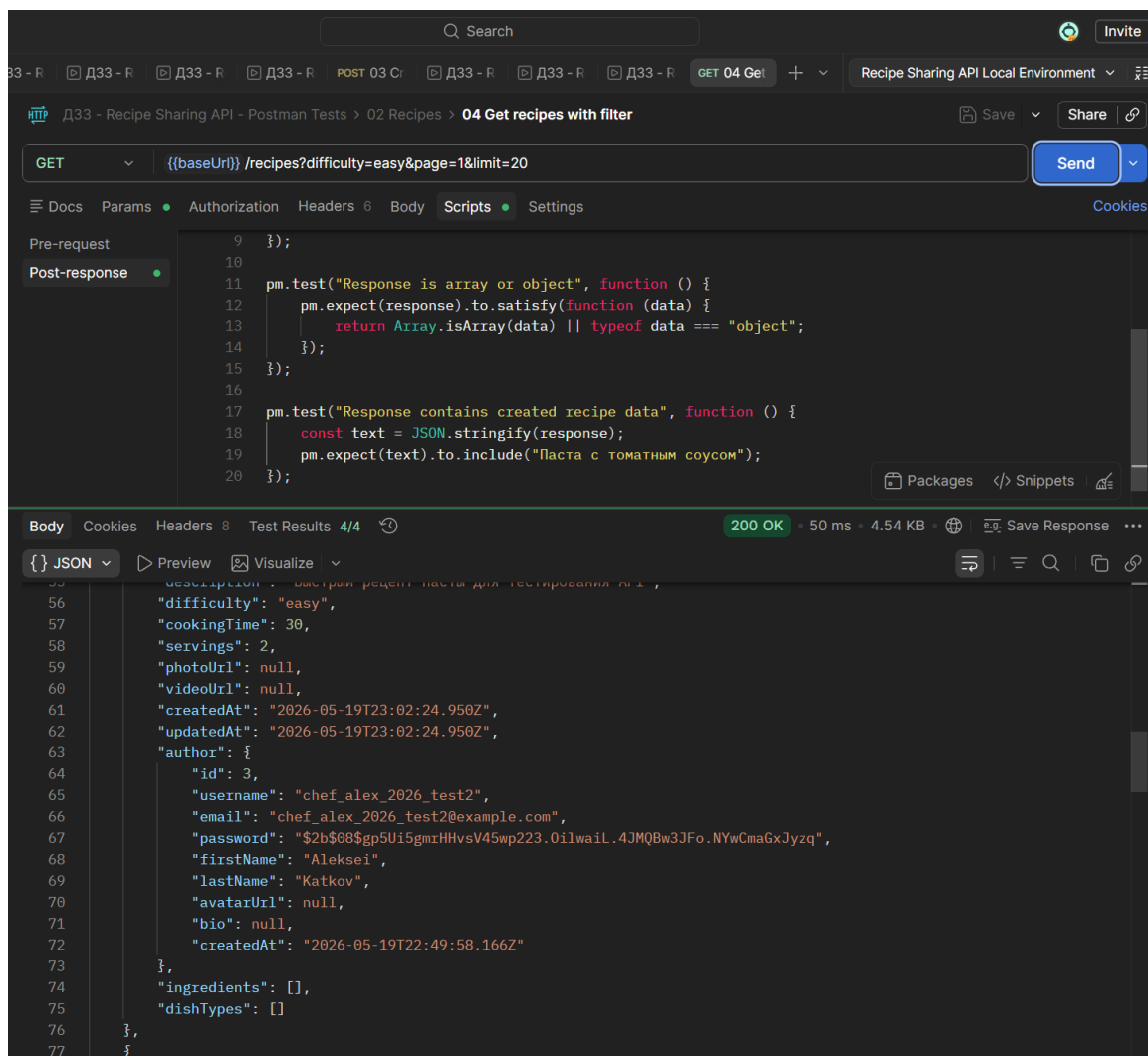


Рис. 4: Получение списка рецептов с фильтрацией

Получение рецепта по идентификатору

Последним шагом сценария выполняется запрос получения данных о рецепте. Для запроса проверяется успешный HTTP-статус, наличие ответа и наличие данных созданного рецепта.

Listing 5: Тесты для получения рецепта по идентификатору

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

const response = pm.response.json();

pm.test("Response exists", function () {
  pm.expect(response).to.exist;
});

pm.test("Response contains recipe data", function () {
  const text = JSON.stringify(response);
  pm.expect(text).to.include("Паста с томатным соусом");
});
```

Результат выполнения запроса представлен на рисунке 5.

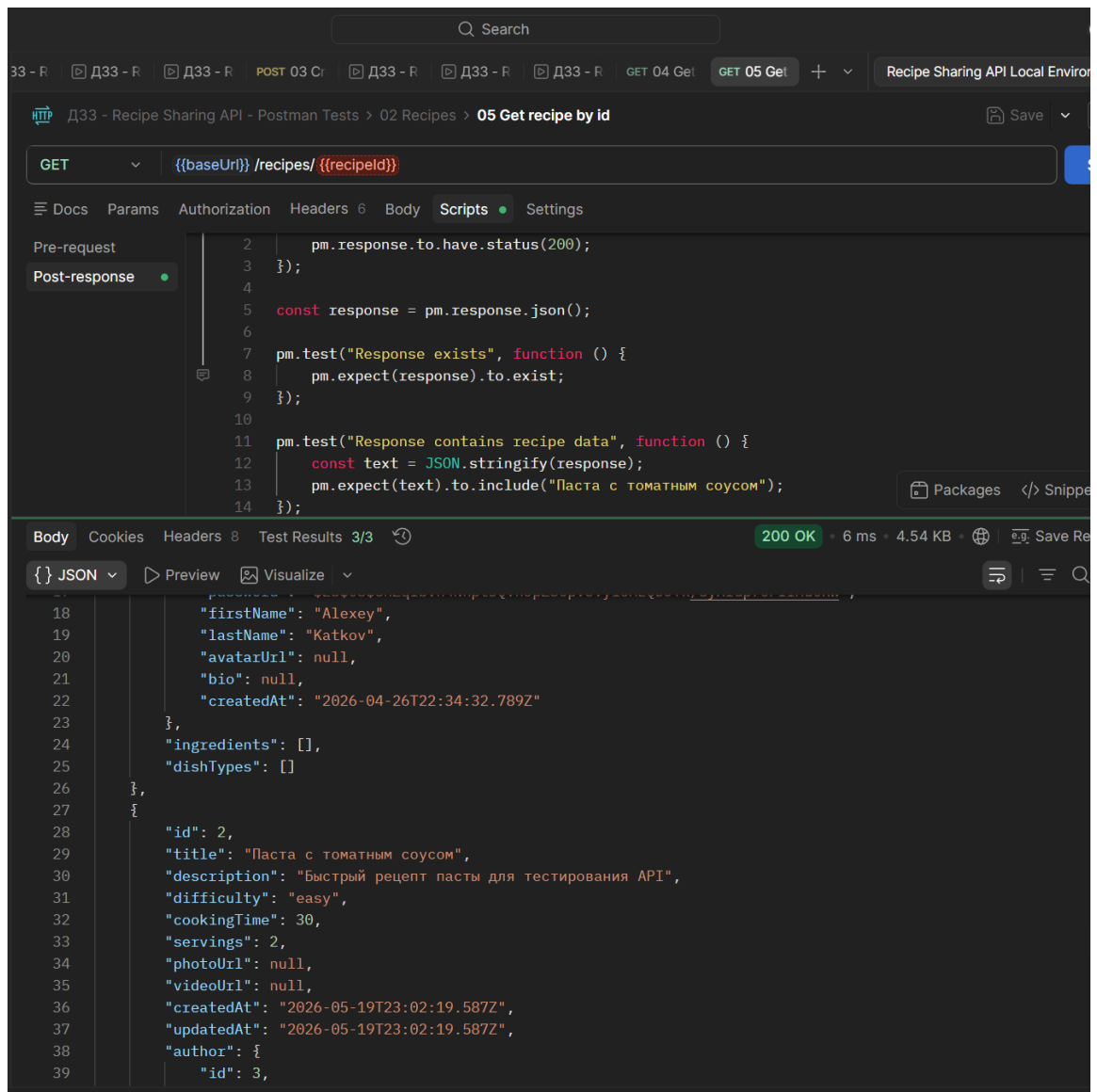


Рис. 5: Получение данных рецепта

Результаты запуска коллекции

После настройки всех запросов коллекция была запущена через Collection Runner. Всего было выполнено 20 проверок. Все проверки завершились успешно: 20 passed, 0 failed, 0 errors.

Результат запуска коллекции представлен на рисунке 6.

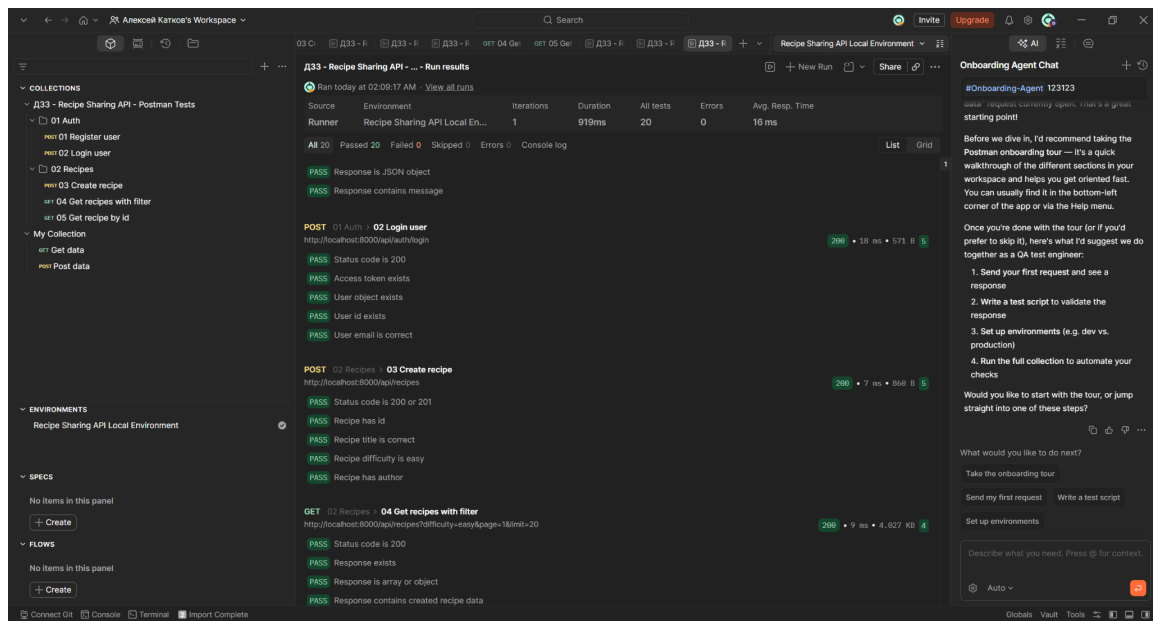


Рис. 6: Успешный запуск коллекции Postman

Вывод

В ходе выполнения домашнего задания было реализовано тестирование API сервиса обмена рецептами средствами Postman. Был создан комплексный пользовательский сценарий, включающий регистрацию пользователя, авторизацию, создание рецепта, получение списка рецептов с фильтрацией и получение данных о рецепте.

В Postman были настроены переменные окружения и переменные коллекции, что позволило автоматически передавать данные между запросами: JWT-токен, идентификатор пользователя и идентификатор созданного рецепта.

Для каждого запроса были написаны автотесты, проверяющие HTTP-статусы, наличие нужных полей в ответе, корректность данных и связность сценария. Итоговый запуск коллекции показал успешное прохождение всех тестов: 20 проверок выполнены без ошибок.

Таким образом, основной процесс работы API был протестирован, а созданная коллекция Postman может использоваться для повторной проверки работоспособности серверной части приложения.